



La Coscienza del Lettore

Sistema di Raccomandazione letture

Corso di Machine Learning

Anno Accademico 2024/2025

Geraldine Montella: 05121 16165
g.montella19@studenti.unisa.it

Repository GitHub: [GuardianMont/LaCoscienzaDelLettore](https://github.com/GuardianMont/LaCoscienzaDelLettore)

Contents

1	Introduction	2
1.1	Scenario considerato	2
1.2	Problema	2
1.3	Task Analizzato	2
2	Dataset	3
2.1	Descrizione del dataset	3
2.2	Caratteristiche Principali	3
2.3	Struttura del Dataset	4
2.4	Arricchimento del Dataset con Google Books API	5
2.5	Conclusione	5
3	Modello di Raccomandazione	6
3.1	Struttura del Modello	6
3.2	Scelte Progettuali	6
3.2.1	Utilizzo del Collaborative Filtering con SVD	6
3.2.2	Pre-elaborazione dei Dati	7
3.2.3	Filtraggio Basato sul Contenuto	7
3.2.4	Personalizzazione della Raccomandazione	8
3.3	Valutazione del Modello	8
3.4	Possibili Miglioramenti	9
3.4.1	Miglioramento della Componentistica del Modello	9
3.4.2	Arricchimento dei Dati	9
3.4.3	Ottimizzazione della Personalizzazione	9
4	Conclusioni	9
5	Interfaccia Grafica	10
5.1	Consiglia Libri	10
5.2	Trova Libri Simili	10
5.3	Libreria Personale	10
5.4	Registrazione Utente	10

1 Introduction

1.1 Scenario considerato

Negli ultimi anni, l'abitudine alla lettura ha subito un calo significativo, complice l'aumento dell'influenza dei social media, delle piattaforme di streaming e della frenesia della vita quotidiana. Molti utenti, pur avendo interesse per i libri, faticano a scegliere cosa leggere o a mantenere una routine di lettura costante.

Le piattaforme di raccomandazione esistenti, come quelle di Goodreads o Amazon Kindle, si concentrano prevalentemente sul suggerire titoli basati su valutazioni generali o tendenze di mercato, senza però adattarsi in modo dinamico alle abitudini di lettura dell'utente. Questo approccio rischia di risultare inefficace per chi ha bisogno di stimoli personalizzati per leggere di più.

1.2 Problema

Il problema affrontato dal progetto è duplice:

1. **Suggerire libri in modo personalizzato:**

Non basta fornire raccomandazioni basate solo sui generi preferiti, ma è necessario considerare anche il ritmo di lettura, i libri già posseduti dall'utente e la varietà dei suggerimenti per evitare ripetitività.

2. **Incentivare la lettura:**

Un sistema di raccomandazione tradizionale non tiene conto delle difficoltà dell'utente nel mantenere un'abitudine costante alla lettura. Questo progetto mira a suggerire titoli che siano accessibili in base al tempo e al numero di pagine che l'utente può realisticamente leggere.

Un semplice sistema di raccomandazione basato sui rating non sarebbe sufficiente a risolvere il problema, poiché non affronta la questione della motivazione alla lettura. Pertanto, è necessario un approccio che combini tecniche avanzate di Collaborative Filtering e Content-Based Filtering, tenendo conto delle specifiche esigenze degli utenti.

1.3 Task Analizzato

Il progetto si concentra sulla creazione di un sistema di raccomandazione intelligente che:

1. Personalizzi le raccomandazioni in base a libri letti, generi preferiti e valutazioni dell'utente.
2. Integri un meccanismo di monitoraggio della lettura, suggerendo libri adatti al ritmo di lettura dell'utente.
3. Diversifichi le raccomandazioni, evitando di proporre sempre libri dello stesso genere per mantenere l'interesse alto.
4. Eviti suggerimenti ridondanti consigliando, se possibile, libri già posseduti dall'utente prima di proporre nuovi acquisti.
5. Sia scalabile e migliorabile, con possibilità di integrare API esterne (Google Books, Goodreads) e feedback dell'utente per affinare le raccomandazioni nel tempo.

L'obiettivo finale è quindi non solo migliorare la qualità dei suggerimenti, ma anche stimolare attivamente la lettura come abitudine regolare.

2 Dataset

2.1 Descrizione del dataset

Il dataset utilizzato in questo progetto è il **Goodbooks-10k**, disponibile su Kaggle al seguente link:

- **Goodbooks-10k Dataset**

Questo dataset rappresenta una delle risorse più complete per la creazione di un sistema di raccomandazione di libri, analogamente a quanto avviene nel settore cinematografico (Netflix, Movielens) e musicale (Million Songs).

2.2 Caratteristiche Principali

Il dataset include:

- **10.000 libri popolari**, con metadati essenziali come titolo, autore, anno di pubblicazione e rating medio.
- **53.424 utenti**, ciascuno con almeno due recensioni, per un totale di circa 10 milioni di valutazioni.
- **Valutazioni dei libri** su una scala da 1 a 5.
- **Lista dei libri "da leggere"** per ogni utente.
- **Tag e categorie** assegnate dagli utenti ai libri.

2.3 Struttura del Dataset

Il dataset è suddiviso in più file CSV, ognuno contenente informazioni specifiche:

ratings.csv Contiene le valutazioni dei libri da parte degli utenti.

Colonne: book_id, user_id, rating

```
book_id,user_id,rating
1,314,5
1,439,3
1,588,5
```

to_read.csv Contiene i libri che gli utenti hanno segnato come “da leggere”.

Colonne: user_id, book_id

```
user_id,book_id
25,3
36,7
42,15
```

books.csv Contiene le informazioni sui libri.

Colonne principali: book_id, title, authors, average_rating, isbn, original_publication_year

```
book_id,title,authors,average_rating,isbn,original_publication_year
1,The Hunger Games,Suzanne Collins,4.34,0439023483,2008
2,Harry Potter and the Sorcerer's Stone,J.K. Rowling,4.47,0439554934,1997
```

book_tags.csv Contiene gli ID dei tag assegnati ai libri.

Colonne: goodreads_book_id, tag_id, count

```
goodreads_book_id,tag_id,count
1,510,220
1,11305,183
2,11557,120
```

tags.csv Fornisce la traduzione degli ID dei tag in nomi comprensibili.

Colonne: tag_id, tag_name

```
tag_id,tag_name
510,young-adult
11305,dystopian
11557,fantasy
```

2.4 Arricchimento del Dataset con Google Books API

Per migliorare l'accuratezza e la completezza delle informazioni sui libri, è stata effettuata un'**imputazione di dati mancanti** tramite l'**API di Google Books**. In particolare, sono stati aggiunti i seguenti campi:

- **Trama (description)**
- **Numero di pagine (pageCount)**
- **Categorie (categories)**

L'imputazione è stata realizzata con una pipeline Python che ha utilizzato più strategie per il recupero dei dati:

1. **Ricerca per ISBN** (se disponibile).
2. **Ricerca per Titolo e Autore.**
3. **Ricerca per Titolo e Anno di Pubblicazione.**

Risultati dell'Arricchimento:

- Oltre il **70%** dei libri ha ottenuto una descrizione completa.
- L'**80%** ha ricevuto il numero di pagine corretto.
- Le **categorie** sono state assegnate in modo efficace al **65%** dei libri.

Esempio di dati dopo l'arricchimento:

```
book_id,title,authors,average_rating,isbn,original_publication_year,description,
pageCount,categories
1,The Hunger Games,Suzanne Collins,4.34,0439023483,2008,"In a dystopian future,
Katniss Everdeen...",374,"Science Fiction, Young Adult"
2,Harry Potter and the Sorcerer's Stone,J.K. Rowling,4.47,0439554934,1997,"Harry
Potter, an orphan, discovers he is a wizard...",309,"Fantasy, Magic, Adventure
"
```

Questo arricchimento ha permesso di:

- Migliorare le raccomandazioni basate su contenuti.
- Aggiungere informazioni fondamentali per un'esperienza utente più completa.
- Fornire dati più precisi per eventuali analisi future.

2.5 Conclusione

Il dataset **Goodbooks-10k**, arricchito con informazioni aggiuntive da Google Books API, costituisce una base solida per lo sviluppo del nostro sistema di raccomandazione. La combinazione di valutazioni, metadati, tag e informazioni arricchite permette di creare suggerimenti altamente personalizzati e migliorare l'esperienza di lettura degli utenti.

3 Modello di Raccomandazione

In questa sezione descriviamo il modello di raccomandazione utilizzato per suggerire libri agli utenti, evidenziando le scelte progettuali adottate e le motivazioni alla base di tali decisioni.

3.1 Struttura del Modello

Il sistema di raccomandazione utilizza un approccio ibrido che combina:

- **Collaborative Filtering** basato su SVD (*Singular Value Decomposition*): Il Collaborative Filtering (o Filtraggio Collaborativo) è una tecnica usata nei sistemi di raccomandazione per suggerire elementi basandosi sulle preferenze espresse da altri utenti. Usando SVD si riduce la matrice [utentixlibri] in un numero più piccolo di "fattori latenti" che rappresentano gusti nascosti, prevede quanto un utente potrebbe gradire un libro anche se non l'ha ancora valutato;
- **Content-Based Filtering** sfruttando la similarità testuale dei libri tramite il metodo TF-IDF (Term Frequency - Inverse Document Frequency), misura quanto è importante una parola in un documento rispetto a tutto il corpo;
- **Filtri personalizzati** basati sulle abitudini di lettura degli utenti.

3.2 Scelte Progettuali

Le decisioni progettuali adottate mirano a garantire un equilibrio tra accuratezza predittiva, efficienza computazionale e personalizzazione dell'esperienza utente.

3.2.1 Utilizzo del Collaborative Filtering con SVD

L'uso della **decomposizione ai valori singolari (SVD)** implementata nella libreria Surprise consente di ridurre la dimensionalità dei dati e identificare relazioni latenti tra utenti e libri. Questo approccio è stato preferito rispetto ai metodi basati su vicinanza (come KNN) in quanto offre una migliore gestione della sparsità dei dati.

```
from surprise import SVD

model = SVD(n_factors=100, n_epochs=20, random_state=42)
model.fit(trainset)
```

Il modello è stato addestrato con una suddivisione *train-test* dell'80%-20%, assicurando una valutazione affidabile delle prestazioni.

3.2.2 Pre-elaborazione dei Dati

Prima di alimentare il modello, i dati vengono filtrati e puliti:

- **Rimozione di valutazioni nulle o pari a zero**, in quanto potrebbero rappresentare errori o mancanza di preferenza reale.
- **Riduzione del numero di utenti**, considerando solo quelli con almeno 5 valutazioni per migliorare l'efficienza computazionale.

```
# Rimozione valutazioni non valide
ratings = ratings[ratings["rating"] > 0]
personal = personal[personal["rating"] > 0]

# Filtro sugli utenti piu attivi
active_users = combined_ratings["user_id"].value_counts()
sampled_users = active_users[active_users > 5].index[:10000]
combined_ratings = combined_ratings[combined_ratings["user_id"].isin(
    sampled_users)]
```

3.2.3 Filtraggio Basato sul Contenuto

Oltre al collaborative filtering, il modello include un sistema di raccomandazione basato sui contenuti. Per determinare la similarità tra i libri, viene utilizzata la tecnica TF-IDF per rappresentare le descrizioni testuali dei libri e successivamente calcolare la similarità del coseno.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

tfidf = TfidfVectorizer(stop_words="english")
tfidf_matrix = tfidf.fit_transform(books["description"].fillna(""))

cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)
```

Le raccomandazioni basate sul contenuto permettono di suggerire libri simili a quelli già apprezzati dall'utente, aumentando la rilevanza dei suggerimenti.

Con il **Term Frequency** si indica quante volte appare una parola in un documento, rispetto alla lunghezza del documento. La componente **Inverse Document Frequency** serve a ridurre il peso delle parole comuni dando più peso alle parole rare ma significative.

3.2.4 Personalizzazione della Raccomandazione

Per personalizzare ulteriormente le raccomandazioni, il modello tiene conto:

- **Dei generi preferiti dall'utente**, determinati analizzando le categorie dei libri precedentemente letti.
- **Delle abitudini di lettura**, in particolare il numero di pagine lette al giorno dall'utente, per raccomandare libri compatibili con il suo ritmo di lettura.

```
def recommend_books(user_id, model, books_df, ratings_df, user_reading_habits, n=10):
    read_books = get_read_books(user_id, ratings_df)
    preferred_genres = get_preferred_genres(user_id, books_df, read_books)
    daily_pages = user_reading_habits.get(user_id, 50)

    candidate_books = books_df[books_df["categories"].isin(preferred_genres)]
    max_pages = daily_pages * 10
    candidate_books = candidate_books[candidate_books["pageCount"] <= max_pages]

    predictions = [(book_id, model.predict(user_id, book_id).est)
                    for book_id in candidate_books["book_id"].unique() if book_id
                    not in read_books]

    predictions.sort(key=lambda x: x[1], reverse=True)
    return [books_df[books_df["book_id"] == book_id]["title"].values[0] for
            book_id, _ in predictions[:n]]
```

3.3 Valutazione del Modello

Il modello è stato valutato utilizzando metriche standard per la qualità della predizione:

- **Root Mean Squared Error (RMSE)**
- **Mean Absolute Error (MAE)**

I risultati ottenuti tramite cross-validation con 5 fold sono i seguenti:

```
RMSE: 0.87
MAE: 0.68
```

Inoltre, il modello SVD è stato confrontato con un approccio basato su KNN e un semplice modello di media globale:

```
RMSE SVD: 0.87
RMSE KNN: 1.03
RMSE Baseline: 1.25
```

Questi risultati confermano che il modello SVD offre una precisione superiore rispetto agli altri metodi testati.

3.4 Possibili Miglioramenti

Pur avendo ottenuto risultati soddisfacenti, ci sono diversi aspetti che potrebbero essere migliorati:

3.4.1 Miglioramento della Componentistica del Modello

- Implementazione di un **modello ibrido più avanzato**, combinando in modo più efficace il Collaborative Filtering con il Content-Based Filtering.
- Utilizzo di **modelli deep learning**, come autoencoder per ridurre la dimensionalità del problema.
- Applicazione di **modelli sequenziali** per tener conto dell'ordine di lettura dei libri.

3.4.2 Arricchimento dei Dati

- Integrazione di nuove fonti di dati, come recensioni testuali o valutazioni dettagliate sugli aspetti dei libri.
- Considerazione di **fattori contestuali** (es. periodo dell'anno o tendenze di lettura).

3.4.3 Ottimizzazione della Personalizzazione

- Miglioramento del **modulo di penalizzazione dei generi ripetitivi**, evitando suggerimenti troppo simili.
- Introduzione di un sistema di **feedback attivo** in cui gli utenti possano segnalare raccomandazioni poco pertinenti, migliorando così la qualità delle future raccomandazioni.

4 Conclusioni

Il sistema sviluppato combina diversi approcci di raccomandazione per offrire suggerimenti personalizzati e accurati agli utenti. I risultati ottenuti mostrano che l'integrazione di Collaborative e Content-Based Filtering consente di migliorare significativamente la qualità delle raccomandazioni rispetto a modelli più semplici. Tuttavia, esistono ancora ampie possibilità di miglioramento, in particolare attraverso l'adozione di modelli più avanzati e l'integrazione di dati più ricchi.

5 Interfaccia Grafica

L'applicazione utilizza **Gradio** per fornire un'interfaccia utente intuitiva e interattiva, organizzata in quattro sezioni principali:

5.1 Consiglia Libri

- L'utente inserisce il proprio ID e sceglie il numero di libri consigliati.
- Il sistema genera suggerimenti basati sulle preferenze dell'utente utilizzando un modello **SVD** per il filtraggio collaborativo.

5.2 Trova Libri Simili

- L'utente inserisce il titolo di un libro.
- Il sistema trova libri simili utilizzando la similarità **coseno**, basata su una rappresentazione **TF-IDF** delle descrizioni.

5.3 Libreria Personale

- Permette di aggiungere libri alla libreria personale.
- L'utente fornisce ID, titolo, valutazione e pagine lette.

5.4 Registrazione Utente

- Un nuovo utente può registrarsi fornendo nome ed email.
- Viene generato un **ID univoco** salvato nel database degli utenti.

L'interfaccia è implementata con **Gradio Blocks**, che consente di navigare tra le diverse sezioni attraverso **Tab** dedicati. L'utente può interagire con i widget (textbox, slider, dropdown) e ricevere risposte in tempo reale. L'avvio dell'interfaccia avviene eseguendo il comando:

```
demo.launch()
```